영지식증명 톺아보기 8편: Halo2와 재귀 증명 - 무한한 계산의 압축

Author: 김대익
Reviewed By: —

Disclaimer: 본 글은 Halo/Halo2와 재귀 증명 기술을 다룹니다. Inner Product Argument, Pasta 곡선 사이클, Accumulation Scheme, 그리고 Nova의 Folding까지 설명합니다.

1. 들어가며: 증명 안의 증명

2019년, Zcash 팀의 Sean Bowe, Jack Grigg, Daira Hopwood가 **Halo**를 발표했습니다[^1].

> "Recursive Proof Composition without a Trusted Setup"

재귀 증명(Recursive Proof)이란?

```
증명 π₁: "나는 계산 C₁을 올바르게 수행했다"
증명 π₂: "나는 π₁을 검증했고, 계산 C₂도 올바르게 수행했다"
증명 π₃: "나는 π₂를 검증했고, 계산 C₃도 올바르게 수행했다"
...
```

최종 증명 하나로 **무한한 계산**을 압축할 수 있습니다!

[^1]: Bowe, S., Grigg, J., Hopwood, D. (2019). *Halo: Recursive Proof Composition without a Trusted Setup*. ePrint 2019/1021. https://eprint.iacr.org/2019/1021

1.1 왜 재귀 증명이 중요한가?

블록체인 확장성:

- 1000개 트랜잭션을 하나의 증명으로
- zkRollup의 핵심 기술
- 무한히 쌓이는 블록 히스토리 압축

증명 집계(Aggregation):

- 여러 독립적 증명을 하나로 합치기
- 배치 검증보다 더 강력

IVC (Incrementally Verifiable Computation):

- 단계별 계산의 점진적 검증
- VDF (Verifiable Delay Function) 구현

1.2 재귀의 난제

단순 접근: "SNARK 검증을 회로로 만들어 SNARK로 증명"

문제점:

1. **페어링 연산**: 회로로 표현하면 ~2500만 제약조건[^2]
2. **비네이티브 산술**: 다른 필드 연산을 시뮬레이션해야 함
3. **Trusted Setup**: Groth16은 회로별 setup 필요

[^2]: Maya ZK. *A Brief Summary of Proof Aggregation*. https://www.maya-zk.com/blog/proof-aggregation

Halo는 이 문제들을 **우회**합니다.

---

2. Inner Product Argument (IPA)

2.1 KZG의 한계

5편에서 다룬 **KZG 커밋먼트**:

- 장점: 상수 크기 증명, 빠른 검증
- 단점: **Trusted Setup 필요**, 페어링 사용

Halo는 **Inner Product Argument (IPA)**를 사용합니다.

2.2 IPA 기본 개념

목표: 벡터 내적 ⟨a, b⟩ = c를 증명

주어진:

- 커밋먼트 P = ⟨a, G⟩ + ⟨b, H⟩ + c·Q
- 여기서 G, H는 생성자 벡터, Q는 추가 생성자

증명자는 a, b를 알고 있음을 증명합니다.

2.3 IPA 프로토콜 (Bulletproofs 스타일)

핵심 아이디어: 재귀적으로 벡터 길이를 반으로 줄이기[^3]

```
길이 n인 벡터 → 길이 n/2인 벡터 → ... → 길이 1인 벡터
```

각 라운드:

1. 벡터를 반으로 나눔: a = (a_L, a_R), b = (b_L, b_R)
2. Cross-term 커밋먼트 계산:
   - L = ⟨a_L, G_R⟩ + ⟨b_R, H_L⟩ + ⟨a_L, b_R⟩·Q
   - R = ⟨a_R, G_L⟩ + ⟨b_L, H_R⟩ + ⟨a_R, b_L⟩·Q
3. 검증자가 랜덤 챌린지 u 제공
4. 새 벡터: a' = a_L + u·a_R, b' = b_L + u^{-1}·b_R
5. 새 생성자: G' = G_L + u^{-1}·G_R, H' = H_L + u·H_R

log(n) 라운드 후, 단일 스칼라로 축소!

[^3]: Bünz, B. et al. (2018). *Bulletproofs: Short Proofs for Confidential Transactions and More*. S&P 2018.

2.4 IPA의 특성

특성	KZG	IPA
Trusted Setup	필요	**불필요**
증명 크기	O(1)	O(log n)
검증 시간	O(1) pairings	O(n) 그룹 연산
가정	d-DLog + Pairing	DLOG만

IPA의 장점: Trusted Setup 제거! **IPA의 단점**: 검증이 선형 시간 (O(n))

---

3. Halo의 핵심 통찰: Accumulation

3.1 문제: 선형 시간 검증

IPA 검증에서 가장 비싼 연산:

```
G = ⟨s, G⟩  (길이 n 다중 스칼라 곱)
```

여기서 s는 라운드 챌린지들 u_1, ..., u_k로 구성된 벡터.

재귀 증명에서 매번 O(n) 연산은 비현실적입니다.

3.2 Halo의 해결책: Accumulation Scheme

핵심 아이디어[^4]:

> "비싼 검증을 미루고, 마지막에 한 번만 수행"

```
증명 π₁ → (부분 검증 + 누적기 acc₁)
증명 π₂ → (부분 검증 + acc₁과 합쳐 acc₂)
...
증명 πₙ → (부분 검증 + acc_{n-1}과 합쳐 accₙ)

최종: accₙ의 완전 검증 (한 번만!)
```

[^4]: Bünz, B., Chiesa, A., Mishra, P., Spooner, N. (2020). *Proof-carrying Data from Accumulation Schemes*. TCC 2020.

3.3 무엇을 누적하는가?

IPA에서 비싼 부분: **G = ⟨s, G⟩** 계산

누적기(Accumulator):

- 여러 증명의 G 계산을 하나로 합침
- 랜덤 선형 결합 사용

```
acc = α₁·G₁ + α₂·G₂ + ... + αₙ·Gₙ
```

부분 검증: O(1) 연산으로 누적이 올바른지 확인 **최종 검증**: 누적된 결과에 대해 한 번의 O(n) 연산

3.4 Accumulation의 수학적 기반

다항식 커밋먼트의 **가산성(Additivity)**:

```
Com(f) + Com(g) = Com(f + g)
```

IPA에서도 커밋먼트가 가산적이므로:

- 여러 Opening Claim을 선형 결합
- 결합된 Claim을 한 번에 검증

---

4. 곡선 사이클 (Cycle of Curves)

4.1 재귀의 필드 문제

문제: 증명을 검증하려면 **타원곡선 연산**이 필요

곡선 E가 체 F_p 위에 정의되면:

- 점 좌표: F_p 원소
- 스칼라 곱: F_q 원소 (q는 곡선 차수)

회로에서 **F_p 산술**을 해야 하는데, 회로의 **네이티브 필드는 F_q**!

→ **비네이티브 산술**: 매우 비효율적 (수백 배 오버헤드)

4.2 곡선 사이클의 해결책

2-사이클(2-cycle): 두 곡선 E_1, E_2 where[^5]:

- E_1의 차수 = E_2의 기반 필드 크기
- E_2의 차수 = E_1의 기반 필드 크기

```
E₁: y² = x³ + b₁  over F_p,  order q
E₂: y² = x³ + b₂  over F_q,  order p
```

[^5]: Electric Coin Company. *The Pasta Curves for Halo 2 and Beyond*. https://electriccoin.co/blog/the-pasta-curves-for-halo-2-and-beyond/

4.3 Pasta 곡선: Pallas와 Vesta

Zcash가 Halo2를 위해 설계한 곡선 쌍[^5]:

Pallas: $y^2 = x^3 + 5$ over F_p **Vesta**: $y^2 = x^3 + 5$ over F_q

여기서:

- $p \approx 2^{254}$ (Pallas 기반 필드)
- $q \approx 2^{254}$ (Vesta 기반 필드, Pallas 차수)
- |Pallas| = q, |Vesta| = p (사이클!)

4.4 사이클의 작동 방식

```
[Pallas 회로]
- F_q 산술 수행
- Vesta 점 연산을 네이티브로 표현 가능!
- 증명 π₁ 생성 (Pallas 커브 점들)

[Vesta 회로]
- F_p 산술 수행
- Pallas 점 연산 (π₁ 검증)을 네이티브로!
- 증명 π₂ 생성 (Vesta 커브 점들)

[다시 Pallas 회로]
- π₂ 검증을 네이티브로...
```

교대로 증명하면서 재귀 구현!

4.5 왜 "Pasta"인가?

Pallas + Vest**a** = Pasta 🍝

- 태양계 소행성 이름 (2 Pallas, 4 Vesta)
- 둘 다 크기가 비슷 (곡선 특성도 유사)
- Gauss가 Pallas 궤도 계산에 FFT 사용 (Halo와 연결!)

---

5. Halo2: PLONK + IPA + Accumulation

5.1 Halo2의 구성

Halo2는 세 가지를 결합합니다[^6]:

1. **PLONKish 산술화**: 7편에서 다룬 PLONK 확장
2. **IPA 기반 다항식 커밋먼트**: Trusted Setup 제거
3. **Accumulation Scheme**: 효율적 재귀

[^6]: Zcash. *The halo2 Book*. https://zcash.github.io/halo2/

5.2 Halo2 회로 구조

```
// Halo2 회로 정의 예시 (개념적)
struct MyCircuit {
    // Advice 열: Witness (비밀)
    a: Column<Advice>,
    b: Column<Advice>,
    c: Column<Advice>,

    // Fixed 열: 상수/셀렉터
    selector: Column<Fixed>,

    // Instance 열: 공개 입력
    public_input: Column<Instance>,
}
```

열 유형:

- **Advice**: 증명자가 채우는 Witness
- **Instance**: 공개 입력/출력
- **Fixed**: 회로에 고정된 상수
- **Selector**: 게이트 활성화 플래그

5.3 Halo2의 Lookup Argument

UltraPLONK의 Lookup을 포함:

```
// "a가 테이블 T에 있음"을 증명
lookup_any("range check", |meta| {
    let a = meta.query_advice(a, Rotation::cur());
    let table = meta.query_fixed(range_table, Rotation::cur());
    vec![(a, table)]
});
```

응용:

- 범위 검사: a ∈ [0, 2^k)
- 비트 분해
- 해시 함수 S-box

5.4 Halo2 성능

메트릭	Groth16	PLONK (KZG)	Halo2 (IPA)
Trusted Setup	회로별	범용	**없음**

메트릭	Groth16	PLONK (KZG)	Halo2 (IPA)
증명 크기	~200 B	~400 B	~1-2 KB
검증 시간	가장 빠름	빠름	중간*

*Halo2는 배치 검증으로 amortize 가능

---

6. Nova: Folding의 혁명

6.1 Accumulation을 넘어서

Halo는 IPA의 비싼 부분만 누적했습니다.

Nova (2021, Srinath Setty)는 질문합니다[^7]:

> "전체 검증을 미룰 수는 없을까?"

[^7]: Kothapalli, A., Setty, S., Tzialla, I. (2022). *Nova: Recursive Zero-Knowledge Arguments from Folding Schemes*. CRYPTO 2022.

6.2 Folding Scheme이란?

Folding: 두 NP 인스턴스를 하나로 "접기"

```
인스턴스 I₁ (witness w₁) + 인스턴스 I₂ (witness w₂)
          ↓ Folding
    인스턴스 I₃ (witness w₃)
```

핵심 성질:

- I_3가 satisfiable ⟺ I_1과 I_2 모두 satisfiable
- Folding 비용 << 두 인스턴스 개별 검증 비용

6.3 R1CS와 Relaxed R1CS

R1CS (3편 복습):

```
(A·z) ⊙ (B·z) = (C·z)
```

문제: R1CS는 바로 Folding 불가!

Relaxed R1CS:

```
(A·z) ⊙ (B·z) = u·(C·z) + E
```

여기서:

- u: 스칼라 (정상 R1CS에서는 u=1)
- E: 에러 벡터 (정상 R1CS에서는 E=0)

6.4 Nova의 Folding

두 Relaxed R1CS 인스턴스:

- (E_1, u_1, x_1) with witness W_1
- (E_2, u_2, x_2) with witness W_2

Folding (랜덤 r 사용):

```
E₃ = E₁ + r·T + r²·E₂
u₃ = u₁ + r·u₂
x₃ = x₁ + r·x₂
W₃ = W₁ + r·W₂
```

여기서 T는 "cross-term" 커밋먼트.

6.5 Nova IVC

IVC (Incrementally Verifiable Computation):

```
F(x₀) = x₁
F(x₁) = x₂
...
F(x_{n-1}) = xₙ
```

Nova 접근:

1. 첫 스텝: π_1 = (Relaxed R1CS for $F(x_0)=x_1$)
2. 스텝 i:
    - 이전 누적 π_{i-1}와 새 스텝 $F(x_{i-1})=x_i$를 **Fold**
    - 결과: π_i

검증자 회로 크기: 상수! (~20,000 제약조건)

6.6 Nova vs Halo 비교

특성	Halo (Accumulation)	Nova (Folding)
무엇을 미룸?	IPA 검증의 일부	전체 검증
검증자 회로	더 큼	~20K 제약조건
재귀 오버헤드	중간	매우 낮음

특성	Halo (Accumulation)	Nova (Folding)
기반	PLONKish + IPA	Relaxed R1CS

---

7. 실제 적용 사례

7.1 Mina Protocol

Mina는 "22KB 블록체인"으로 유명[^8]:

- 전체 블록체인 상태를 하나의 증명으로
- Pickles (Halo 스타일 재귀) 사용
- Pasta 곡선 사용

장점:

- 풀노드가 전체 히스토리 저장 불필요
- 모바일에서도 검증 가능

[^8]: Mina Protocol. https://minaprotocol.com/

7.2 Scroll

Scroll은 Ethereum zkRollup:

- Halo2 기반 zkEVM
- EVM 바이트코드를 직접 증명
- PSE (Privacy and Scaling Explorations) 포크 사용

7.3 Taiko

Taiko도 Halo2 기반 zkRollup:

- Type-1 zkEVM (완전 Ethereum 호환)
- 다중 증명자 시스템

7.4 SP1/Risc0

STARK + SNARK 하이브리드:

```
STARK 증명 (빠른 prover)
    ↓ 재귀 압축
Groth16/PLONK 증명 (작은 온체인)
```

SP1, Risc0 등의 zkVM이 이 패턴 사용.

---

8. 재귀 증명의 미래

8.1 SuperNova

SuperNova (2022)[^9]:

- Nova를 **다중 명령어**로 확장
- 서로 다른 회로들의 IVC
- VM (Virtual Machine) 증명에 적합

[^9]: Kothapalli, A., Setty, S. (2022). *SuperNova: Proving Universal Machine Executions without Universal Circuits*. ePrint 2022/1758.

8.2 HyperNova

HyperNova:

- 고차 다항식 지원
- CCS (Customizable Constraint Systems) 기반
- 더 표현력 있는 제약 시스템

8.3 ProtoStar

ProtoStar:

- PLONK를 위한 Folding
- Nova의 아이디어를 PLONKish로

8.4 앞으로의 방향

```
[현재]
STARK → Groth16 wrap (하이브리드)

[미래?]
직접적인 효율적 재귀
+ 양자 저항
+ 더 작은 증명
```

---

9. 마치며: 증명의 압축은 어디까지?

Halo와 재귀 증명이 열어준 세계:

기술적 성과:

- **Trusted Setup 제거**: IPA + Accumulation
- **효율적 재귀**: 곡선 사이클
- **극한의 압축**: Folding Scheme

실용적 영향:

- 22KB 블록체인 (Mina)
- zkEVM (Scroll, Taiko)
- 범용 zkVM (SP1, Risc0)

개념적 통찰:

- "비싼 것을 미루면 공짜가 된다" (Amortization)
- "두 문제를 하나로 접으면 절반이 된다" (Folding)
- "곡선을 교대하면 네이티브가 된다" (Cycles)

다음 편에서는 **STARK와 FRI**를 다룹니다:

- 해시 기반 증명의 원리
- Reed-Solomon 코드와 근접성 테스트
- 양자 저항과 투명성

시리즈 로드맵

1편: 비밀을 증명한다는 것 ✓
2편: 컴퓨터가 이해하는 수학 ✓
3편: 프로그램을 다항식으로 - 산술화 ✓
4편: Zcash Sprout - 최초의 프라이버시 암호화폐 ✓
5편: Groth16의 마법 ✓
6편: Filecoin의 PoRep - 저장 증명 ✓
7편: PLONK - 범용 SNARK의 시대 ✓
8편: Halo2와 재귀 증명 (현재) ✓
9편: STARK와 FRI
10편: zkEVM과 zkVM
...

Appendix A: IPA 프로토콜 상세

A.1 Setup

공개 파라미터:

- G = (G_0, G_1, ..., G_{n-1}): 랜덤 생성자들
- H = (H_0, H_1, ..., H_{n-1}): 랜덤 생성자들
- Q: 추가 생성자

A.2 Commit

벡터 a, b와 내적 c = ⟨a, b⟩에 대해:

```
P = ⟨a, G⟩ + ⟨b, H⟩ + c·Q
```

A.3 Prove (재귀적)

라운드 i (벡터 길이 2^k):

1. a = (a_L, a_R), b = (b_L, b_R) 분할

2. L_i = ⟨a_L, G_R⟩ + ⟨b_R, H_L⟩ + ⟨a_L, b_R⟩·Q
3. R_i = ⟨a_R, G_L⟩ + ⟨b_L, H_R⟩ + ⟨a_R, b_L⟩·Q
4. 챌린지 u_i ← 해시(transcript)
5. a' = a_L + u_i·a_R
6. b' = u_i^{-1}·b_L + b_R
7. G' = G_L + u_i^{-1}·G_R
8. H' = u_i·H_L + H_R

k 라운드 후: 스칼라 a, b, c 공개

A.4 Verify

1. 모든 (L_i, R_i, u_i) 사용해 P' 재구성
2. G = ⟨s, G⟩ 계산 (가장 비싼 부분!)
3. P' = a·G + b·H + (a·b)·Q 확인

Appendix B: Pasta 곡선 파라미터

Pallas

```
방정식: y² = x³ + 5
기반 필드: F_p where
p = 0x40000000000000000000000000000000224698fc094cf91b992d30ed00000001
차수: q = 0x40000000000000000000000000000000224698fc0994a8dd8c46eb2100000001
```

Vesta

```
방정식: y² = x³ + 5
기반 필드: F_q (Pallas 차수)
차수: p (Pallas 기반 필드)
```

사이클 속성

```
|Pallas| = q = Vesta의 기반 필드
|Vesta| = p = Pallas의 기반 필드
```

References

1. Bowe, S., Grigg, J., Hopwood, D. (2019). *Halo: Recursive Proof Composition without a Trusted Setup*. ePrint 2019/1021. https://eprint.iacr.org/2019/1021
2. Maya ZK. *A Brief Summary of Proof Aggregation*. https://www.maya-zk.com/blog/proof-aggregation
3. Bünz, B. et al. (2018). *Bulletproofs: Short Proofs for Confidential Transactions and More*. S&P 2018.

4. Bünz, B., Chiesa, A., Mishra, P., Spooner, N. (2020). *Proof-carrying Data from Accumulation Schemes*. TCC 2020.
5. Electric Coin Company. *The Pasta Curves for Halo 2 and Beyond*. https://electriccoin.co/blog/the-pasta-curves-for-halo-2-and-beyond/
6. Zcash. *The halo2 Book*. https://zcash.github.io/halo2/
7. Kothapalli, A., Setty, S., Tzialla, I. (2022). *Nova: Recursive Zero-Knowledge Arguments from Folding Schemes*. CRYPTO 2022. https://eprint.iacr.org/2021/370
8. Mina Protocol. https://minaprotocol.com/
9. Kothapalli, A., Setty, S. (2022). *SuperNova: Proving Universal Machine Executions without Universal Circuits*. ePrint 2022/1758.
10. Electric Coin Company. *Explaining Halo 2*. https://electriccoin.co/blog/explaining-halo-2/
11. Kudelski Security. *On the Security of Halo2 Proof System*. https://kudelskisecurity.com/research/on-the-security-of-halo2-proof-system
12. Veridise. *Recursive SNARKs and IVC*. https://veridise.com/blog/learn-blockchain/recursive-snarks-and-incrementally-verifiable-computation-ivc/
13. Lambda Class. *Incrementally Verifiable Computation: Nova*. https://blog.lambdaclass.com/incrementally-verifiable-computation-nova/
14. Anoma Research. *Demystifying Recursive Zero-Knowledge Proofs*. https://anoma.net/research/demystifying-recursive-zero-knowledge-proofs